

Assignment 4: 16-Bit ALU

Ziad Mostafa Abdelaziz

July 15, 2026

1 Introduction

ALU_TOP is the fundamental building block of the processor, which is responsible for carrying out different functions:

- Signed Arithmetic functions through the **ARITHMETIC_UNIT** block.
- Logic functions through the **LOGIC_UNIT** block.
- Shift functions through the **SHIFT_UNIT** block.
- Comparison functions through the **CMP_UNIT** block.

The **Decoder Unit** is responsible for enabling which function to operate according to the most significant 2-bits of the ALU_FUNC control bus, specifically ALU_FUNC[3:2].

2 Specifications

- All Outputs are registered.
- All registers are cleared using an Asynchronous active low reset.
- **Arith_OUT Width**: The output bus width is parameterized as $\text{OUT_DATA_WIDTH} = 2 * \text{DATA_WIDTH}$ (default 32 bits) to support full-precision outcomes during multiplication ($16 \times 16 \rightarrow 32$ -bit precision signed result), preventing truncations or overflows.
- **Arith_Flag** is activated "High" only when the ALU performs one of the arithmetic operations (Signed Addition, Signed Subtraction, Signed Multiplication, Signed Division) and is "LOW" otherwise.
- **Logic_Flag** is activated "High" only when the ALU performs Boolean operations (AND, OR, NAND, NOR).
- **CMP_Flag** is activated "High" only when the ALU performs Comparison operations (Equal, Greater than, Less than) or NOP.
- **Shift_Flag** is activated "High" only when the ALU performs shifting operations (shift right, shift left).

3 ALU Function Tables

3.1 Decoder Truth Table

ALU_FUNC[3:2]	Arith_En	Logic_En	CMP_En	Shift_En
00	1	0	0	0
01	0	1	0	0
10	0	0	1	0
11	0	0	0	1

Table 1: Decoder Enable Logic

3.2 ALU_FUNC Operation

ALU_FUNC	Operation	ALU_OUT (when applicable)
0000	Arithmetic: Signed Addition	—
0001	Arithmetic: Signed Subtraction	—
0010	Arithmetic: Signed Multiplication	—
0011	Arithmetic: Signed Division	—
0100	Logic: AND	—
0101	Logic: OR	—
0110	Logic: NAND	—
0111	Logic: NOR	—
1000	NOP	Equal to 0
1001	CMP: A = B	Equal to 1 (else 0)
1010	CMP: A > B	Equal to 2 (else 0)
1011	CMP: A < B	Equal to 3 (else 0)
1100	SHIFT: A $\gg$ 1	—
1101	SHIFT: A $\ll$ 1	—
1110	SHIFT: B $\gg$ 1	—
1111	SHIFT: B $\ll$ 1	—

Table 2: ALU Function Selection

4 Simulation Waveforms & Analysis

The simulation verifies the operation of the ALU over various test cases utilizing a 100 kHz clock with a 40% low (4000 ns) and 60% high (6000 ns) duty cycle.

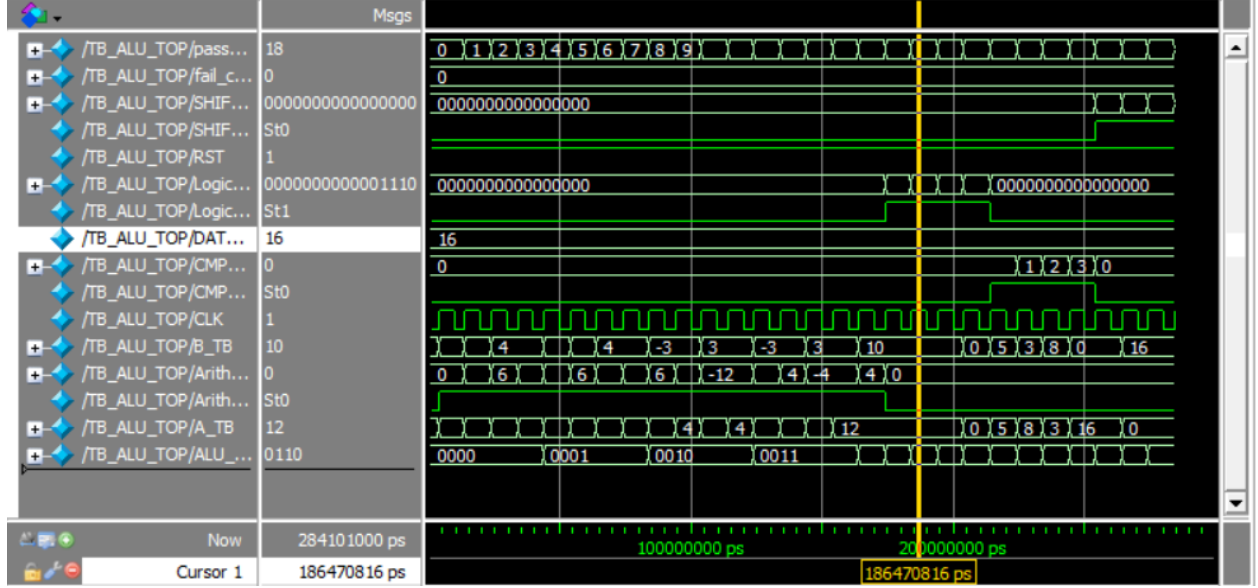


Figure 1: Full Simulation Waveforms of the 16-Bit ALU

4.1 Waveform Analysis & Commentary

- **Reset Behavior:** At $t = 0$, the reset signal (RST) is held active low. The internal registers are cleared, meaning all outputs and flags are set to 0. Operations only begin properly once RST goes high at 100 ns. As all outputs are synchronized with the positive edge of the clock (CLK), we see that the updated calculation results appear safely after the active clock edge.
- **Negative Arithmetic Examples:** To deal with negative values in the testbench, inputs were provided in a signed context using their two's complement form. For instance, the decimal value -4 produces the two's complement hexadecimal `FFFC`, and -10 gives `FFF6`. The testbench applied operations like `ADD_NEG_NEG` ($A = -4, B = -10$), which produced -14 in the arithmetic output and successfully asserted `Arith_Flag` high. The next clock edge safely latched these items.
- **Shift and Logic Transitions:** For Logical AND (`0100`), applying $A = 12$ (`000C`) and $B = 10$ (`000A`) produces the result 8, updating the `Logic_OUT` output properly while asserting the `Logic_Flag` completely decoupled from the Arithmetic Flag. Furthermore, logical shifts like $A \ll 1$ (where $A = 16$) effectively multiplied the input by 2, yielding 32 as verified on the `SHIFT_OUT` bus.
- **Comparison Output:** For a comparison operation like $A > B$ (`1010`), providing $A = 8$ and $B = 3$ assigns the output value 2 to `CMP_OUT`, acting as a status code.

5 Simulation Output Logs

The simulation log validates each function's expected value versus its actual registered output and corresponding module flag, resulting in 28 PASSED test cases and 0 FAILED out of 28.

```

# =====
# ALU_TOP Testbench - 28 Test Cases
# =====
# PASS | ADD_NEG_NEG | A=-4 B=-10 FUNC=0000 -> OUT=-14 FLAG=1
# PASS | ADD_POS_NEG | A=10 B=-4 FUNC=0000 -> OUT=6 FLAG=1
# PASS | ADD_NEG_POS | A=-10 B=4 FUNC=0000 -> OUT=-6 FLAG=1
# PASS | ADD_POS_POS | A=10 B=4 FUNC=0000 -> OUT=14 FLAG=1
# PASS | SUB_NEG_NEG | A=-4 B=-10 FUNC=0001 -> OUT=6 FLAG=1
# PASS | SUB_POS_NEG | A=10 B=-4 FUNC=0001 -> OUT=14 FLAG=1
# PASS | SUB_NEG_POS | A=-10 B=4 FUNC=0001 -> OUT=-14 FLAG=1
# PASS | SUB_POS_POS | A=10 B=4 FUNC=0001 -> OUT=6 FLAG=1
# PASS | MUL_NEG_NEG | A=-4 B=-3 FUNC=0010 -> OUT=12 FLAG=1
# PASS | MUL_POS_NEG | A=4 B=-3 FUNC=0010 -> OUT=-12 FLAG=1
# PASS | MUL_NEG_POS | A=-4 B=3 FUNC=0010 -> OUT=-12 FLAG=1
# PASS | MUL_POS_POS | A=4 B=3 FUNC=0010 -> OUT=12 FLAG=1
# PASS | DIV_NEG_NEG | A=-12 B=-3 FUNC=0011 -> OUT=4 FLAG=1
# PASS | DIV_POS_NEG | A=12 B=-3 FUNC=0011 -> OUT=-4 FLAG=1
# PASS | DIV_NEG_POS | A=-12 B=3 FUNC=0011 -> OUT=-4 FLAG=1
# PASS | DIV_POS_POS | A=12 B=3 FUNC=0011 -> OUT=4 FLAG=1
# PASS | LOGIC_AND | A=12 B=10 FUNC=0100 -> OUT=8 FLAG=1
# PASS | LOGIC_OR | A=12 B=10 FUNC=0101 -> OUT=14 FLAG=1
# PASS | LOGIC_NAND | A=12 B=10 FUNC=0110 -> OUT=-9 FLAG=1
# PASS | LOGIC_NOR | A=12 B=10 FUNC=0111 -> OUT=-15 FLAG=1
# PASS | NOP | A=0 B=0 FUNC=1000 -> OUT=0 FLAG=1
# PASS | CMP_EQUAL | A=5 B=5 FUNC=1001 -> OUT=1 FLAG=1
# PASS | CMP_GREATER | A=8 B=3 FUNC=1010 -> OUT=2 FLAG=1
# PASS | CMP_LESS | A=3 B=8 FUNC=1011 -> OUT=3 FLAG=1
# PASS | SHIFT_A_RIGHT | A=16 B=0 FUNC=1100 -> OUT=8 FLAG=1
# PASS | SHIFT_A_LEFT | A=16 B=0 FUNC=1101 -> OUT=32 FLAG=1
# PASS | SHIFT_B_RIGHT | A=0 B=16 FUNC=1110 -> OUT=8 FLAG=1
# PASS | SHIFT_B_LEFT | A=0 B=16 FUNC=1111 -> OUT=32 FLAG=1
# =====
# RESULTS: 28 PASSED, 0 FAILED (out of 28)
# =====
# ** Note: $finish      : E:/courses/Digital IC design Course/ASIC Flow/My Work/My Assignments/Assignment 4 Verilog/TB_ALU_TOP.v(170)
# Time: 284101 ns Iteration: 0 Instance: /TB_ALU_TOP

```

Figure 2: Simulation Log Output Screen Shot